Numerical algorithms

Outline

Introduction

Change of base

Primality test

Sieve of Eratosthenes

Greatest common divisor

In summary

Introduction

Four classical algorithms on natural numbers

In this chapter we study four classical algorithms that work on natural numbers:

- the **change of base**, which writes a natural number given in base ten as a sequence of digits in another base b;
- the **primality test**, which decides whether a natural number is prime;
- the **Sieve of Eratosthenes**, which finds all the prime numbers up to a given natural number;
- the **Euclidean algorithm**, which computes the greatest common divisor of two natural numbers.

One elementary operation

- All four algorithms are built on one elementary operation, the **division with remainder**:
  - the change of base reads the digits as remainders;
  - the primality test and the sieve look for remainders equal to zero;
  - the Euclidean algorithm replaces a pair of numbers with a smaller pair through a remainder.

- For each algorithm we give the pseudo-code (with arrays indexed from 1) and an execution step by step

- Under each pseudo-code, the links «Run in Colab» open its implementation in Python, C++ and Java

Change of base

Introduction: Change of base

- We know how to read a sequence of digits in base b as a number: now we do the **opposite**
- Given a number in base ten, we compute its digits in any base $b \geq 2$ (for instance, the binary digits used by a computer)
- The tool is the division with remainder: one division extracts one digit
- We prove why this works, write the algorithm, and run it on four numbers

From digits to numbers, and back

- In the chapter «Representation of numbers» we saw that a sequence of k digits $(d_{k-1} \dots d_1 d_0)_b$ in base b represents the natural number

$$\sum_{i=0}^{k-1} d_i \, b^i$$

- We also saw that $\lfloor \log_b n \rfloor + 1$ digits are needed to write a natural number $n \geq 1$ in base b

- In this section we solve the **inverse problem**: given n in base ten, we compute its digits in base b

Definition 1: division with remainder

*Given a natural number n and a natural number $b \geq 1$, the **division with remainder** (or Euclidean division) of n by b computes the two natural numbers q and r such that*

$$n = q\ b + r \qquad \text{with} \qquad 0 \leq r < b.$$

*The number n is called the **dividend**, b the **divisor**, q the **quotient** and r the **remainder**. The two numbers q and r are unique, and we write*

$$q = \lfloor n/b \rfloor \qquad \text{and} \qquad r = n \bmod b.$$

The key fact

One division by b extracts
the **least significant digit** of n in base b,

and leaves as quotient the number
formed by the **remaining digits**.

Observation 1

Let $n \geq 1$ be a natural number whose representation in base $b \geq 2$ is $(d_{k-1} \ldots d_1 d_0)_b$. Then

$$n \bmod b = d_0 \qquad \text{and}$$

$$\lfloor n/b \rfloor = \sum_{i=1}^{k-1} d_i \, b^{i-1} = (d_{k-1} \ldots d_1)_b,$$

where the quotient is 0 if $k = 1$.

Proof

Collecting the factor b from all the terms with $i \geq 1$, we obtain

$$n = \sum_{i=0}^{k-1} d_i \, b^i = \left(\sum_{i=1}^{k-1} d_i \, b^{i-1} \right) b + d_0.$$

- Since d_0 is a digit in base b, we have $0 \leq d_0 < b$
- By the uniqueness of the quotient and of the remainder of the division with remainder, the quotient of n divided by b is $\sum_{i=1}^{k-1} d_i \, b^{i-1}$ and the remainder is d_0



Repeating the division

- Dividing n by b, the remainder is the least significant digit d_0, and the quotient is the number written with the other digits $(d_{k-1} \dots d_1)_b$
- Dividing the quotient again by b gives d_1, and so on
- Repeating the division until the quotient is 0 produces all the digits of n in base b, from the least significant to the most significant one, in exactly $\lfloor \log_b n \rfloor + 1$ divisions

The problem

Given a natural number $n > 0$ represented in base 10 (decimal)
and a target base $b \geq 2$,
we consider the problem of converting n to base b.

- Next: the pseudo-code of the algorithm to convert sequences of digits from base ten into base b

Algorithm: ChangeOfBase (1/2)

Input : A natural number $n \geq 1$ (in base 10) and a base $b \geq 2$

Output : An array $d[1..nd]$ of digits in base b and the number of digits nd

```
1 function ChangeOfBase(n, b)
2 |   nd := ⌊log_b n⌋ + 1        // Number of digits needed
3 |   d := new array[1..nd]
4 |   k := 1
```

Algorithm: ChangeOfBase (2/2)

function *ChangeOfBase(n, b)* (continued)

```
5   while n > 0 do
6       q := ⌊n/b⌋                                  // Quotient
7       r := n mod b    // Remainder (digit in base b)
8       d[k] ← r
9       n ← q
10      k ← k + 1
11  end
12  return d, nd
13 end
```

Run in Colab: Python · C++ · Java

How ChangeOfBase works (1/2)

- The algorithm calculates the number of digits nd needed to represent n in base b using the formula $\lfloor \log_b n \rfloor + 1$
- It initializes an array $d[1..nd]$ to store the digits and a counter k to track the current position
- The main loop repeatedly divides n by b to extract the digits from right to left (from the least significant to the most significant one)
- At each iteration, the remainder $r = n \bmod b$ gives the next digit, which is stored in $d[k]$

How ChangeOfBase works (2/2)

- The quotient $q = \lfloor n/b \rfloor$ becomes the new value of n for the next iteration
- The algorithm terminates when $n = 0$, returning the array d and the total number of digits nd
- The digits in d are stored in **reverse order**: the entry $d[k]$ is the digit d_{k-1} of weight b^{k-1}, so
  - $d[1] = d_0$ is the least significant digit (rightmost)
  - $d[nd] = d_{nd-1}$ is the most significant digit (leftmost)

Let us now run the algorithm on four numbers:
7 and 8 in base 2, 25 in base 3, 100 in base 8.

Example (1/4)

Translation of 7 into base $b = 2$:

iteration k	n	n/b	q	r	digit
1	7	$7/2$	3	1	$d[1] = d_0$ (LSD)
2	3	$3/2$	1	1	$d[2] = d_1$
3	1	$1/2$	0	1	$d[3] = d_2$ (MSD)

Accordingly, 7 in base 2 is $(111)_2$, with $nd = \lfloor \log_2 7 \rfloor + 1 = 3$ digits.

Example (2/4)

Translation of 8 into base $b = 2$:

iteration k	n	n/b	q	r	digit
1	8	$8/2$	4	0	$d[1] = d_0$ (LSD)
2	4	$4/2$	2	0	$d[2] = d_1$
3	2	$2/2$	1	0	$d[3] = d_2$
4	1	$1/2$	0	1	$d[4] = d_3$ (MSD)

Accordingly, 8 in base 2 is $(1000)_2$, with $nd = \lfloor \log_2 8 \rfloor + 1 = 4$ digits.

Example (3/4)

Translation of 25 into base $b = 3$:

iteration k	n	n/b	q	r	digit
1	25	$25/3$	8	1	$d[1] = d_0$ (LSD)
2	8	$8/3$	2	2	$d[2] = d_1$
3	2	$2/3$	0	2	$d[3] = d_2$ (MSD)

Accordingly, 25 in base 3 is $(221)_3$, with $nd = \lfloor \log_3 25 \rfloor + 1 = 3$ digits.

Example (4/4)

Translation of 100 into base $b = 8$:

iteration k	n	n/b	q	r	digit
1	100	$100/8$	12	4	$d[1] = d_0$ (LSD)
2	12	$12/8$	1	4	$d[2] = d_1$
3	1	$1/8$	0	1	$d[3] = d_2$ (MSD)

Accordingly, 100 in base 8 is $(144)_8$, with $nd = \lfloor \log_8 100 \rfloor + 1 = 3$ digits.

Summary

Change of base:

- The remainder of the division of n by b is the least significant digit of n in base b; the quotient is the number written with the other digits
- The algorithm performs one division for each digit: $\lfloor \log_b n \rfloor + 1$ iterations
- The digits are produced from the least significant to the most significant one, so the result is read backwards: from $d[nd]$ to $d[1]$

Primality test

Introduction: Primality test

- Prime numbers are the building blocks of the natural numbers, and deciding whether a number is prime is a basic task (for instance, in cryptography)
- We start from a direct algorithm suggested by the definition
- Then we prove that it suffices to test the candidate divisors up to $\sqrt{n}$: far fewer divisions

When the remainder is zero

- The change of base uses the remainder n mod b to read a digit
- A remainder equal to zero has a meaning of its own: it says that the division is exact, that is, that b is a **divisor** of n
- Counting divisors leads to the **prime numbers**

Definition 2: divisor

*A natural number $d \geq 1$ is a **divisor** of a natural number n if there exists a natural number k such that $n = k\ d$:*

$$d \text{ is a divisor of } n \iff \exists\, k \in \mathbb{N},\ n = k\ d$$

or equivalently, if $n \bmod d = 0$.

Definition 3: prime number

*A natural number $n \geq 2$ is **prime** if its only divisors are 1 and n itself. A natural number $n \geq 2$ which is not prime is called **composite**: it has a divisor d with $2 \leq d \leq n-1$. The numbers 0 and 1 are not prime.*

The problem

Given a natural number n,
we consider the problem of determining
whether n is a prime number.

A first algorithm

- The definition suggests a direct algorithm: test, one by one, the candidate divisors of n different from 1 and n
- The following observation shows that the candidates larger than $n/2$ can be skipped

Observation 2

For every natural number $n \geq 2$, if d is a divisor of n with $d < n$, then $d \leq n/2$.

Proof

- Since d is a divisor of n, there exists a natural number k such that $n = k\ d$
- Since $d < n$, we have $k \geq 2$, hence
$$d = \frac{n}{k} \leq \frac{n}{2}.$$

□

- It follows that it suffices to check whether any natural number d in $\{2, 3, \ldots, \lfloor n/2 \rfloor\}$ is a divisor of n
- If no such divisor exists, n is prime

Algorithm: BasicPrimalityTest

Input : A natural number n
Output : `true` if n is prime, `false` otherwise

```
1  function BasicPrimalityTest(n)
2      if n < 2 then
3          return false          // 0 and 1 are not prime
4      end
5      for d := 2 to ⌊n/2⌋ do
6          if n mod d = 0 then
7              return false      // d divides n:  not prime
8          end
9      end
10     return true               // No divisor found:  n is prime
11 end
```

Run in Colab: Python · C++ · Java

How BasicPrimalityTest works

- Numbers less than 2 are not prime by definition
- The loop tests every candidate divisor d from 2 up to $\lfloor n/2 \rfloor$
- If d is a divisor of n (i.e., $n \bmod d = 0$), the function returns `false` immediately
- If the loop completes without finding a divisor, n is prime and the function returns `true`

- For a prime number the loop is executed completely: for $n = 29$ the algorithm tests the 13 candidates $d = 2, 3, \ldots, 14$
- We now show that far fewer candidates are enough

Testing the divisors up to $\sqrt{n}$

The divisors of a natural number come in pairs:
d and n/d.

In each pair, one of the two numbers is at most $\sqrt{n}$.

Observation 3

For every natural number $n \geq 2$, if d is a divisor of n then n/d is a divisor of n.

Proof

If d is a divisor of n, then there exists a natural number k such that

$$n = k\ d.$$

Since $k = n/d$, it follows that n/d is a natural number.
Accordingly,

$$n = d\left(\frac{n}{d}\right),$$

so n/d is also a divisor of n. □

Observation 4

For every natural number $n \geq 2$, if $d > \sqrt{n}$ then $n/d < \sqrt{n}$.

Proof

If $d > \sqrt{n}$, dividing n by both sides of this inequality, we obtain

$$\frac{n}{d} < \frac{n}{\sqrt{n}} = \sqrt{n}.$$

Therefore, n/d is smaller than $\sqrt{n}$. □

Corollary 1

Every composite natural number $n \geq 2$ has a divisor d with $2 \leq d \leq \lfloor \sqrt{n} \rfloor$. Equivalently, a natural number $n \geq 2$ is prime if and only if no natural number in $\{2, 3, \ldots, \lfloor \sqrt{n} \rfloor\}$ is a divisor of n.

Proof

- Since n is composite, it has a divisor d with $2 \leq d \leq n-1$
- If $d \leq \sqrt{n}$, then $d \leq \lfloor \sqrt{n} \rfloor$, because d is a natural number, and we are done
- Otherwise $d > \sqrt{n}$: by the two previous observations, n/d is a divisor of n with $n/d < \sqrt{n}$
- Moreover $n/d \geq 2$ because $d < n$
- Hence n/d is a divisor of n with $2 \leq n/d \leq \lfloor \sqrt{n} \rfloor$



Divisors come in pairs

- From the previous observations, for every natural number $n \geq 2$, if $d > \sqrt{n}$ is a divisor of n, then n/d is also a divisor of n and $n/d < \sqrt{n}$
- It follows that every divisor of n greater than $\sqrt{n}$ is paired with a divisor of n smaller than $\sqrt{n}$
- Therefore, if n has any divisor at all (other than 1 and n), at least one of them must be $\leq \sqrt{n}$

Example

Let us consider $n = 36$: $\sqrt{36} = 6$. The divisors of 36 other than 1 and 36 are $2, 3, 4, 6, 9, 12, 18$.

divisor $d > \sqrt{36}$	paired divisor $36/d < \sqrt{36}$
9	$36/9 = 4$
12	$36/12 = 3$
18	$36/18 = 2$

Every divisor > 6 is paired with a divisor < 6; the divisor $6 = \sqrt{36}$ is paired with itself.

A faster algorithm

- By the corollary, the loop of the basic algorithm can stop at $\lfloor \sqrt{n} \rfloor$ instead of $\lfloor n/2 \rfloor$
- Next: the pseudo-code of the Primality Test algorithm

Algorithm: PrimalityTest

Input : A natural number n
Output : `true` if n is prime, `false` otherwise

```
1  function PrimalityTest(n)
2      if n < 2 then
3          return false          // 0 and 1 are not prime
4      end
5      d := 2
6      while d ≤ ⌊√n⌋ do
7          if n mod d = 0 then
8              return false      // d divides n:  not prime
9          end
10         d ← d + 1
11     end
12     return true               // No divisor found:  n is prime
13 end
```

Run in Colab: Python · C++ · Java

How PrimalityTest works

- Numbers less than 2 are not prime by definition
- The loop tests every candidate divisor d from 2 up to $\lfloor \sqrt{n} \rfloor$
- If d divides n (i.e., $n \bmod d = 0$), the function returns `false` immediately
- If the loop completes without finding a divisor, then, by the corollary, n is prime and the function returns `true`
- For $n = 2$ and $n = 3$ we have $\lfloor \sqrt{n} \rfloor = 1 < 2$: the loop is not executed and the function returns `true`

Example (1/3)

Check $n = 36$: $\lfloor \sqrt{36} \rfloor = 6$

d	36 mod d	action
2	0	return `false`
36 is not prime $(36 = 2 \cdot 18)$.		

Sapienza Università di Roma

Example (2/3)

Check $n = 29$: $\lfloor \sqrt{29} \rfloor = 5$

d	29 mod d	action
2	$1 \neq 0$	continue
3	$2 \neq 0$	continue
4	$1 \neq 0$	continue
5	$4 \neq 0$	continue
Loop ends. Return `true`: **29 is prime**.		

Example (3/3)

Check $n = 25$: $\lfloor \sqrt{25} \rfloor = 5$

d	25 mod d	action
2	$1 \neq 0$	continue
3	$1 \neq 0$	continue
4	$1 \neq 0$	continue
5	0	return `false`
25 is not prime $(25 = 5 \cdot 5)$.		

For the prime number 29 the algorithm tests 4 candidates, while the basic algorithm tests 13.

Summary

Primality test:

- The divisors of n come in pairs d and n/d, and in each pair one of the two numbers is at most $\sqrt{n}$
- Hence $n \geq 2$ is prime if and only if no natural number in $\{2, 3, \ldots, \lfloor \sqrt{n} \rfloor\}$ divides it
- The Primality Test executes at most $\lfloor \sqrt{n} \rfloor - 1$ iterations, instead of the $\lfloor n/2 \rfloor - 1$ iterations of the basic algorithm

Sieve of Eratosthenes

Introduction: Sieve of Eratosthenes

- Now we want **all** the prime numbers up to n, not only the answer for one number
- The sieve is one of the oldest algorithms known, and a simple way to build the table of all the primes up to n
- We prove two properties that tell where the marking starts and where it stops, then we run the algorithm for $n = 10$ and $n = 20$

Crossing out multiples

- The Primality Test answers the question for one number
- When we need all the prime numbers up to n, calling it on $2, 3, \ldots, n$ repeats many divisions
- The sieve works the other way around: instead of searching for the divisors of each number, it **crosses out the multiples** of each prime

The problem

Given a natural number n,
we consider the problem of determining
all prime numbers smaller than or equal to n.

- The **Sieve of Eratosthenes** is an ancient algorithm (attributed to the Greek mathematician Eratosthenes of Cyrene, c. 276–194 BC) that computes all prime numbers up to a natural number n
- The key idea is to start with all natural numbers from 2 to n marked as potentially prime, and then iteratively mark the multiples of each prime as composite

Definition 4: multiple

A natural number m is a ***multiple*** *of a natural number $p \geq 1$ if there exists a natural number k such that $m = k\ p$:*

$$m \text{ is a multiple of } p \iff \exists\, k \in \mathbb{N},\ m = k\ p$$

or equivalently, if $m \bmod p = 0$.

Where the marking starts

The first observation tells where the marking of the multiples of p can start.

Observation 5

For every natural number $p \geq 2$, if a natural number m is a multiple of p with $p < m < p^2$, then m has a divisor d with $2 \leq d < p$.

Proof

- Since m is a multiple of p, there exists a natural number d such that $m = d\ p$. Then d is a divisor of m
- Moreover, since $m > p$ and $m = d\ p$, we have $d \geq 2$
- Since $m < p^2$ and $m = d\ p$, we have $d < p$
- Hence $2 \leq d < p$



Start marking at p^2

- From the previous observation, for every prime p and every multiple m of p with $p < m < p^2$, the number m has a divisor d with $2 \leq d < p$
- Hence m also has a prime divisor q with $2 \leq q < p$, and was therefore already marked as composite when q was processed
- Thus, when marking the multiples of p, one can start with p^2

Example

Let $p = 5$. Its multiples between p and $p^2 = 25$ are $m = 10, 15, 20$.

multiple m	$m = d \cdot p$	divisor $d < p$	already marked when
10	$10 = 2 \cdot 5$	$d = 2$	$p = 2$ (from $j = 4$, step 2)
15	$15 = 3 \cdot 5$	$d = 3$	$p = 3$ (from $j = 9$, step 3)
20	$20 = 4 \cdot 5$	$d = 4$	$p = 2$ (from $j = 4$, step 2)

- The number 20 has the divisor $d = 4$, which is not prime, and also the prime divisor 2, which marked it
- The next multiple is $25 = 5 \cdot 5 = p^2$, which has not been marked by any previous prime number
- Therefore, for $p = 5$, one can start marking numbers from 25

Where the outer loop stops

The second observation tells where the outer loop over p can stop.

Observation 6

For every natural number $n \geq 2$, every composite natural number $m \leq n$ has a prime divisor q with $q \leq \lfloor \sqrt{n} \rfloor$.

Proof

- By the corollary of the section «Primality test», m has a divisor d with
$$2 \leq d \leq \lfloor \sqrt{m} \rfloor \leq \lfloor \sqrt{n} \rfloor$$
- Let q be the smallest divisor of m with $q \geq 2$; then $q \leq d \leq \lfloor \sqrt{n} \rfloor$
- The number q is prime: if q had a divisor c with $2 \leq c < q$, then c would also be a divisor of m (from $q = k_1 c$ and $m = k_2 q$ we get $m = k_2 k_1 c$), against the choice of q as the smallest one



Stop at $\lfloor \sqrt{n} \rfloor$

- Hence, after the multiples of all the primes $p \leq \lfloor \sqrt{n} \rfloor$ have been marked, every composite number up to n is marked, and the outer loop can stop
- Next: the pseudo-code of the Sieve of Eratosthenes

Algorithm: SieveOfEratosthenes (1/2)

Input : A natural number $n \geq 1$
Output : A boolean array $prime[1..n]$ where $prime[i] = 1$ if and only if i is prime

```
1 function SieveOfEratosthenes(n)
2 |   prime := new array[1..n]    // declare boolean array
3 |   prime ← 1                   // initialize all entries to 1
4 |   prime[1] ← 0                // 1 is not prime
5 |   p := 2
```

Algorithm: SieveOfEratosthenes (2/2)

function *SieveOfEratosthenes(n)* (continued)

```
6     while p ≤ ⌊√n⌋ do
7         if prime[p] = 1 then
8             j := p²
9             while j ≤ n do
10                prime[j] ← 0                    // j not prime
11                j ← j + p
12            end
13        end
14        p ← p + 1
15    end
16    return prime
17 end
```

Run in Colab: Python · C++ · Java

How SieveOfEratosthenes works (1/2)

- The array *prime* is declared and all entries initialized to 1; the entry *prime*[1] is immediately set to 0, since 1 is not prime
- The outer loop iterates over each candidate prime p from 2 up to $\lfloor \sqrt{n} \rfloor$
- If *prime*$[p] = 1$, then p is prime (a composite p would have been marked by its smallest prime divisor), and all its multiples starting from p^2 are set to 0

How SieveOfEratosthenes works (2/2)

- The inner loop starts at p^2 because all smaller multiples $2p, 3p, \ldots, (p-1)p$ have already been set to 0 when processing the smaller primes $2, 3, 5, 7, \ldots$
- After the execution of the algorithm, $prime[i] = 1$ if and only if i is prime

Example (1/2)

- We run the sieve with $n = 10$, so $\lfloor \sqrt{10} \rfloor = 3$
- Each row of the table shows the state of the array $prime[1..10]$ after the assignment $prime[j] \leftarrow 0$
- **Blue cells** indicate entries just set to 0
- **Red cells** indicate the current value of p

Example (2/2)

		prime[1..10]										
p	j	1	2	3	4	5	6	7	8	9	10	action
–	–	0	1	1	1	1	1	1	1	1	1	initialization
2	4	0	1	1	0	1	1	1	1	1	1	$prime[4] \leftarrow 0$
	6	0	1	1	0	1	0	1	1	1	1	$prime[6] \leftarrow 0$
	8	0	1	1	0	1	0	1	0	1	1	$prime[8] \leftarrow 0$
	10	0	1	1	0	1	0	1	0	1	0	$prime[10] \leftarrow 0$
	12											$j = 12 > 10$: stop
3	9	0	1	1	0	1	0	1	0	0	0	$prime[9] \leftarrow 0$
	12											$j = 12 > 10$: stop
Outer loop ends: $p = 4 > \lfloor \sqrt{10} \rfloor = 3$.												

Result: the prime numbers ≤ 10 are 2, 3, 5, 7.

Example (1/5)

- We run the sieve with $n = 20$, so $\lfloor \sqrt{20} \rfloor = 4$
- Each row of the table shows the state of the array $prime[1..20]$ after the assignment $prime[j] \leftarrow 0$
- **Blue cells**: entry just set to 0; **red cells**: current value of p
- When the entry is already 0, the assignment does not change the array
- The array is too long for one slide: we show the entries 1..10 and then the entries 11..20

Example (2/5)

The prime $p = 2$, entries $prime[1..10]$:

p	j	$prime[1..10]$										action
		1	2	3	4	5	6	7	8	9	10	
–	–	0	1	1	1	1	1	1	1	1	1	initialization
2	4	0	1	1	0	1	1	1	1	1	1	$prime[4] \leftarrow 0$
	6	0	1	1	0	1	0	1	1	1	1	$prime[6] \leftarrow 0$
	8	0	1	1	0	1	0	1	0	1	1	$prime[8] \leftarrow 0$
	10	0	1	1	0	1	0	1	0	1	0	$prime[10] \leftarrow 0$
	12	0	1	1	0	1	0	1	0	1	0	$prime[12] \leftarrow 0$
	14	0	1	1	0	1	0	1	0	1	0	$prime[14] \leftarrow 0$
	16	0	1	1	0	1	0	1	0	1	0	$prime[16] \leftarrow 0$
	18	0	1	1	0	1	0	1	0	1	0	$prime[18] \leftarrow 0$
	20	0	1	1	0	1	0	1	0	1	0	$prime[20] \leftarrow 0$
	22											$j = 22 > 20$: stop

Example (3/5)

The prime $p = 2$, entries $prime[11..20]$:

p	j	$prime[11..20]$										action
		11	12	13	14	15	16	17	18	19	20	
–	–	1	1	1	1	1	1	1	1	1	1	initialization
2	4	1	1	1	1	1	1	1	1	1	1	$prime[4] \leftarrow 0$
	6	1	1	1	1	1	1	1	1	1	1	$prime[6] \leftarrow 0$
	8	1	1	1	1	1	1	1	1	1	1	$prime[8] \leftarrow 0$
	10	1	1	1	1	1	1	1	1	1	1	$prime[10] \leftarrow 0$
	12	1	0	1	1	1	1	1	1	1	1	$prime[12] \leftarrow 0$
	14	1	0	1	0	1	1	1	1	1	1	$prime[14] \leftarrow 0$
	16	1	0	1	0	1	0	1	1	1	1	$prime[16] \leftarrow 0$
	18	1	0	1	0	1	0	1	0	1	1	$prime[18] \leftarrow 0$
	20	1	0	1	0	1	0	1	0	1	0	$prime[20] \leftarrow 0$
	22											$j = 22 > 20$: stop

Example (4/5)

The values $p = 3$ and $p = 4$, entries $prime[1..10]$:

p	j	$prime[1..10]$ 1	2	3	4	5	6	7	8	9	10	action
3	9	0	1	1	0	1	0	1	0	0	0	$prime[9] \leftarrow 0$
	12	0	1	1	0	1	0	1	0	0	0	$prime[12] \leftarrow 0$ (already 0)
	15	0	1	1	0	1	0	1	0	0	0	$prime[15] \leftarrow 0$
	18	0	1	1	0	1	0	1	0	0	0	$prime[18] \leftarrow 0$ (already 0)
	21											$j = 21 > 20$: stop
4												$prime[4] = 0$: skip

Outer loop ends: $p = 5 > \lfloor \sqrt{20} \rfloor = 4$.

Example (5/5)

The values $p = 3$ and $p = 4$, entries $prime[11..20]$:

p	j	$prime[11..20]$ 11	12	13	14	15	16	17	18	19	20	action
3	9	1	0	1	0	1	0	1	0	1	0	$prime[9] \leftarrow 0$
	12	1	0	1	0	1	0	1	0	1	0	$prime[12] \leftarrow 0$ (already 0)
	15	1	0	1	0	0	0	1	0	1	0	$prime[15] \leftarrow 0$
	18	1	0	1	0	0	0	1	0	1	0	$prime[18] \leftarrow 0$ (already 0)
	21											$j = 21 > 20$: stop
4												$prime[4] = 0$: skip
Outer loop ends: $p = 5 > \lfloor\sqrt{20}\rfloor = 4$.												

Result: the prime numbers ≤ 20 are 2, 3, 5, 7, 11, 13, 17, 19.

Summary

Sieve of Eratosthenes:

- The sieve does not divide: it marks as composite the multiples of each prime p
- The marking of the multiples of p starts at p^2: the smaller multiples have a smaller prime divisor and are already marked
- The outer loop stops at $\lfloor \sqrt{n} \rfloor$: every composite number up to n has a prime divisor not larger than $\lfloor \sqrt{n} \rfloor$

Greatest common divisor

Introduction: Greatest common divisor

- The last algorithm of the chapter deals with **two numbers at once**: we look for the divisors they have in common
- The greatest common divisor is used, for instance, to reduce a fraction to lowest terms
- We prove one property that replaces a pair of numbers with a smaller pair, and the **Euclidean algorithm** follows from it
- At the end we use it to decide whether two numbers are coprime

Definition 5: common divisor and greatest common divisor (1/2)

Let n, m be two natural numbers with $n, m \geq 1$.

- *A natural number $d \geq 1$ is a **common divisor** of n and m if d is a divisor of both n and m, that is, if there exist natural numbers k_1, k_2 such that*

$$n = k_1\ d \qquad \text{and} \qquad m = k_2\ d.$$

Equivalently, $n \bmod d = 0$ and $m \bmod d = 0$.

Definition: common divisor and greatest common divisor (2/2)

- *The **greatest common divisor** of n and m, denoted* $\gcd(n, m)$*, is the largest common divisor of n and m.*

Existence of the greatest common divisor

- The greatest common divisor always exists: 1 is a common divisor, and every common divisor is at most $\min\{n, m\}$
- For instance, the common divisors of 48 and 18 are $1, 2, 3, 6$, hence $\gcd(48, 18) = 6$

Given two natural numbers n, m with $n, m \geq 1$,
we consider the problem of computing
their greatest common divisor.

A smaller pair

The following observation replaces the pair (n, m) with a pair of smaller numbers which has the same greatest common divisor.

Observation 7

For every pair of natural numbers $n \geq 1$ and $m \geq 1$,

$$\gcd(n, m) = \gcd(m,\ n \bmod m).$$

Proof (1/3)

Let $q = \lfloor n/m \rfloor$, so that $n = qm + (n \bmod m)$.

Let d_1 be a common divisor of n and m. Then there exist natural numbers k_1, k_2 such that

$$n = k_1 d_1 \qquad \text{and} \qquad m = k_2 d_1.$$

Since $n \bmod m = n - qm$, we obtain

$$n \bmod m = (k_1 - qk_2)\, d_1,$$

where $k_1 - qk_2 \geq 0$ because $n \bmod m \geq 0$. So d_1 is also a divisor of $n \bmod m$. Hence d_1 is a common divisor of m and $n \bmod m$.

Proof (2/3)

Conversely, let d_2 be a common divisor of m and $n \bmod m$. Then there exist natural numbers k_1, k_2 such that

$$m = k_1 d_2 \qquad \text{and} \qquad n \bmod m = k_2 d_2.$$

Since $n = qm + (n \bmod m)$, we obtain

$$n = (qk_1 + k_2)\, d_2,$$

so d_2 is also a divisor of n. Hence d_2 is a common divisor of n and m.

Proof (3/3)

Therefore, n and m have exactly the same common divisors as m and $n \bmod m$, and hence

$$\gcd(n, m) = \gcd(m,\ n \bmod m).$$

In the right-hand side, $\gcd(m, 0)$ is understood as the largest divisor of m (every $d \geq 1$ divides $0 = 0 \cdot d$), that is, m itself. $\square$

Euclidean algorithm

- The **Euclidean algorithm** (attributed to the ancient Greek mathematician Euclid, c. 300 BC) computes $\gcd(n, m)$ efficiently by repeatedly replacing the pair (n, m) with $(m,\ n \bmod m)$
- The algorithm does not need $n \geq m$: if $n < m$, then $n \bmod m = n$, and the first replacement simply swaps the two numbers, turning (n, m) into (m, n)

Why the replacement works

- From the previous observation, replacing (n, m) with $(m,\ n \bmod m)$ does not change the greatest common divisor
- Since $n \bmod m < m$, the second argument strictly decreases at each replacement
- Therefore, the algorithm terminates when the remainder becomes 0; at that point, the greatest common divisor is the other argument:

$$\gcd(n, 0) = n.$$

Algorithm: EuclideanAlgorithm

Input : Two natural numbers $n, m \geq 1$
Output: $\gcd(n, m)$

```
1 function EuclideanAlgorithm(n, m)
2     while m ≠ 0 do
3         r := n mod m
4         n ← m
5         m ← r
6     end
7     return n                    // m = 0:  n is the GCD
8 end
```

Run in Colab: Python · C++ · Java

How EuclideanAlgorithm works

- At each step, (n, m) is replaced by $(m,\ n \bmod m)$
- By the previous observation, this replacement preserves the value of $\gcd(n, m)$: denoting by n_0 and m_0 the input values, at every iteration $\gcd(n, m) = \gcd(n_0, m_0)$
- Since $n \bmod m < m$, the value of m strictly decreases at every iteration, so the algorithm terminates
- When $m = 0$, the equality $\gcd(n, m) = \gcd(n_0, m_0)$ together with $\gcd(n, 0) = n$ gives the result

Example (1/2)

Compute $\gcd(48, 18)$:

step	n	m	$r = n \bmod m$
1	48	18	$48 \bmod 18 = 12$
2	18	12	$18 \bmod 12 = 6$
3	12	6	$12 \bmod 6 = 0$
4	6	0	— (loop ends)

Result: $\gcd(48, 18) = 6$.

Example (2/2)

Compute $\gcd(252, 105)$:

step	n	m	$r = n \bmod m$
1	252	105	$252 \bmod 105 = 42$
2	105	42	$105 \bmod 42 = 21$
3	42	21	$42 \bmod 21 = 0$
4	21	0	— (loop ends)

Result: $\gcd(252, 105) = 21$.

Coprimality

Definition 6: coprime numbers

Two natural numbers $n, m \geq 1$ *are said to be* ***coprime*** *(or* ***relatively prime****) if they share no common divisor other than* 1*:*

$$n \text{ and } m \text{ are coprime} \iff \gcd(n, m) = 1.$$

- Since the Euclidean algorithm computes $\gcd(n, m)$, to test coprimality it suffices to call the algorithm and check whether the result is equal to 1

Example

- Compute $\gcd(35, 12)$: the Euclidean algorithm returns 1. Therefore, 35 and 12 are **coprime**
- Compute $\gcd(48, 18)$: the Euclidean algorithm returns 6. Therefore, 48 and 18 are **not coprime**

Summary

Greatest common divisor:

- The pairs (n, m) and $(m,\ n \bmod m)$ have the same common divisors, hence the same greatest common divisor
- The Euclidean algorithm repeats the replacement until the second number is 0, and returns the first one, since $\gcd(n, 0) = n$
- Two numbers are coprime if and only if the Euclidean algorithm returns 1

In summary

Summary (1/2)

Division with remainder: $n = q\,b + r$ with $0 \leq r < b$; all four algorithms are built on it

Change of base:

- each division by b gives the next digit as remainder, from the least significant one
- $\lfloor \log_b n \rfloor + 1$ divisions; the digits are read backwards

Primality test:

- the divisors come in pairs d and n/d: testing up to $\lfloor \sqrt{n} \rfloor$ is enough
- at most $\lfloor \sqrt{n} \rfloor - 1$ iterations instead of $\lfloor n/2 \rfloor - 1$

Summary (2/2)

Sieve of Eratosthenes:

- marks the multiples of each prime p, starting at p^2
- the outer loop stops at $\lfloor \sqrt{n} \rfloor$

Euclidean algorithm:

- $\gcd(n, m) = \gcd(m,\ n \bmod m)$, until the second number is 0; then $\gcd(n, 0) = n$
- n and m are coprime if and only if it returns 1